

GeomVLA: Unifying Scene, Motion, and Action in 3D

Zi Yin Xiong¹ Nikos Gkanatsios¹ Moritz Reuss² Katerina Fragkiadaki¹

¹Carnegie Mellon University ²NVIDIA
{ziyinx,katef}@andrew.cmu.edu

Abstract: We present **GeomVLA**, a Vision-Language-Action (VLA) model that unifies perception, latent scene motion prediction, and action generation within a shared robot-centric 3D coordinate frame. Our approach lifts pretrained VLM features into spatially grounded 3D scene tokens using depth and camera calibration, while retaining the semantic representations learned during VLM pretraining. We further introduce a *3D Scene Trajectory Denoiser*, a task-conditioned module that learns a latent representation of how scene points are expected to move in 3D. Rather than executing the predicted trajectory as an open-loop plan, GeomVLA extracts intermediate motion tokens from the trajectory denoiser and uses them to condition a 3D flow-based action denoiser through geometry-aware attention. GeomVLA achieves state-of-the-art performance on CALVIN, competitive performance on LIBERO and RoboTwin2.0, and outperforms strong baselines in real-world manipulation settings without robot-action pretraining. Extensive ablations show that future-motion reasoning alone is insufficient: the primary gains are associated with maintaining geometric consistency among scene representation, motion prediction, and robot actions throughout the perception-to-action pipeline. Project webpage: ziyin-xiong.github.io/geomvla.io.

Keywords: Vision-Language-Action Models, 3D Motion Prediction

1 Introduction

Vision-language-action (VLA) models [1, 2, 3] have recently emerged as a promising paradigm for language-conditioned robot manipulation by leveraging the broad semantic knowledge of pretrained vision-language models (VLMs) [4, 5, 6]. However, most existing VLAs operate primarily on 2D images while robot manipulation fundamentally requires reasoning about geometry, contact, and motion in 3D space [7, 8, 3]. This creates a mismatch between image-centric representations and the metric geometry required for control.

One natural solution is to represent both the scene and robot actions directly in metric 3D coordinates. Recent 3D manipulation policies show that explicit geometric representations improve spatial reasoning and data efficiency [9, 10, 11, 12, 13]. More recent VLA systems combine such geometric representations with pretrained VLMs through lifted 3D features, spatial positional encodings, or dual-stream architectures [14, 15, 16, 17, 18, 19]. Although these approaches improve geometric grounding, they primarily focus on static 3D scene representations and action generation, without addressing how intermediate future reasoning should be represented and coupled to control.

A complementary line of work introduces future prediction as an intermediate reasoning process for robot manipulation [20, 21, 22, 23, 24, 25, 26]. Instead of mapping observations directly to actions, these methods encourage policies to reason about how the scene will evolve through predicted videos, optical flow, visual traces, or trajectories. Such representations can expose task dynamics and support long-horizon planning. Many of these methods perform future reasoning in pixel or image space, whereas actions are executed in 3D. We argue that considering 3D representation and future prediction separately leaves a mismatch among scene understanding, motion reasoning, and control (Figure 1). Consequently, these components do not interact through a common geometric representation.

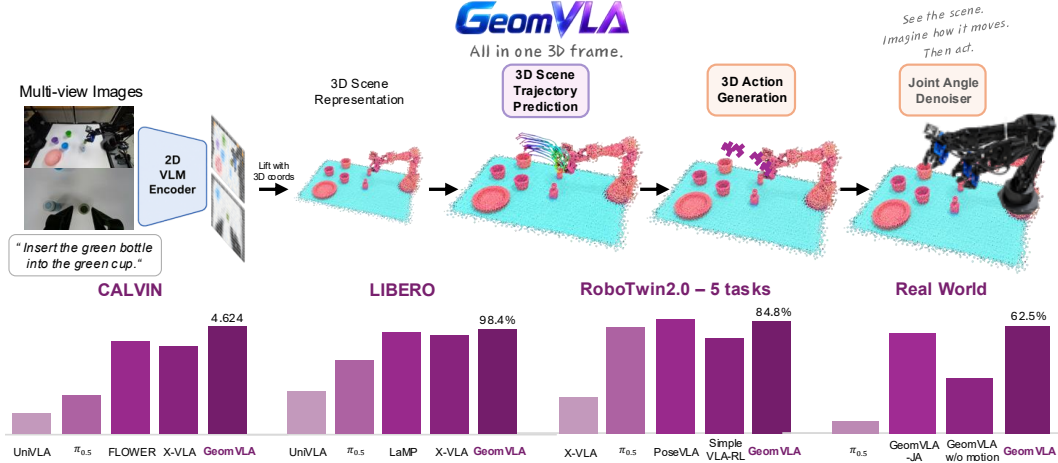


Figure 1: **GeomVLA enables motion-guided 3D visual-language-action learning by bridging perception, latent future motion, and robot control in a shared 3D space.** It lifts pretrained 2D VLM features into a geometrically grounded 3D representation, predicts latent 3D scene trajectories as an intermediate reasoning signal, and conditions a 3D action denoiser on the resulting scene-motion representation to generate robot actions. This design improves performance in both simulation and the real world. It also supports joint-angle action denoising by conditioning on scene-motion and end-effector pose-action features.

In this work, we introduce **GeomVLA**, a 3D motion-aware VLA that unifies perception, latent future-motion reasoning, and action generation within a shared robot-centric 3D coordinate frame. We hypothesize that future-motion reasoning is most useful when perception, motion prediction, and control share the same geometry. GeomVLA therefore expresses scene features, predicted motion, and Cartesian actions in a common robot-centric metric frame. Concretely, GeomVLA first lifts pretrained VLM features into spatially grounded 3D scene tokens using depth and camera calibration. We then introduce a *3D Scene Trajectory Denoiser*, a task-conditioned module that predicts future 3D point motion and exposes intermediate motion tokens to the action policy. Rather than executing a predicted trajectory directly, the action policy uses intermediate trajectory-denoiser features as a latent geometric reasoning signal. Consequently, scene features, motion representations, and robot trajectories interact within a unified metric coordinate frame throughout action generation.

We evaluate GeomVLA across CALVIN [27], LIBERO [28], RoboTwin2.0 [29], and real-world manipulation benchmarks. Our approach achieves state-of-the-art performance on CALVIN, competitive performance on LIBERO and RoboTwin2.0, and outperforms strong baselines in real-world settings without requiring robot-action pretraining. Extensive ablations show that future-motion reasoning alone is insufficient: the primary gains are associated with maintaining geometric consistency between scene representation, motion prediction, and robot actions throughout the perception-to-action pipeline.

In summary, our contributions are as follows:

- We introduce GeomVLA, a VLA framework that unifies scene representation, latent future-motion reasoning, and action generation within a shared 3D coordinate frame.
- We propose a *3D Scene Trajectory Denoiser*, a task-conditioned module that predicts future scene motion directly in 3D space and provides intermediate motion features to downstream action generation through geometry-aware attention.
- We achieve state-of-the-art performance on CALVIN, competitive performance on LIBERO and RoboTwin2.0, and strong real-world results without robot-action pretraining.
- Extensive ablations show that maintaining geometric consistency between scene representation, motion prediction, and robot actions benefits long-horizon manipulation.

2 Related Work

3D Vision-Language-Action Models. Recent 3D manipulation policies [10, 9, 30, 11, 31, 32, 33, 34] improve spatial reasoning and data efficiency by representing scene observations and robot actions within a shared geometric space. More recent 3D VLA models [14, 15, 18, 17, 35, 36, 37, 38, 39] further incorporate pretrained VLM semantics into such frameworks. Existing approaches generally either inject 3D representations directly into pretrained VLM backbones through 3D positional encodings or couple VLM features with separate 3D encoders for action prediction. While these designs improve geometric grounding, they primarily focus on aligning scene representation and action generation, without explicitly addressing how intermediate reasoning processes should be represented within the control pipeline. In contrast, our method unifies scene representation, future-motion reasoning, and action generation within the same 3D coordinate frame.

Motion Priors for Visuomotor Policies. Recent visuomotor policies increasingly use future prediction as an auxiliary learning signal. Prior work explores video prediction [40, 41, 20, 42, 43], optical flow or visual traces [44, 21, 26, 22, 24, 45, 46, 25, 47], object/gripper trajectories [23, 48, 49, 50, 51, 52], and depth [53, 54] to provide temporal cues beyond direct action supervision. These methods differ primarily in how future motion is represented and how it interacts with downstream control. Some approaches use explicit plan-then-act formulations [49, 22], where predicted trajectories or flow fields are generated before policy execution, while others incorporate motion prediction as a latent supervisory signal within the policy itself [46]. These approaches thus predominantly reason in pixel or image-space representations, even when actions are ultimately executed in 3D. In contrast, our method represents future scene motion directly within the same robot-centric 3D coordinate frame used for scene representation and action generation.

3 Method

Figure 2 illustrates GeomVLA. Given multi-view RGB-D observations $\{(I_v, D_v)\}_{v=1}^V$, a language instruction ℓ , and robot proprioception $\mathbf{p}$, the model lifts VLM-contextualized features into 3D using depth, camera calibration, and 3D positional encoding (§3.1). A 3D Scene Trajectory Denoiser then learns an instruction-conditioned latent representation of future point motion (§3.2). Its intermediate motion tokens condition a flow-based 3D action denoiser through geometry-aware attention, producing an action chunk $A = (a_1, \dots, a_T)$ (§3.3). Scene positions, predicted point motion, and Cartesian action tokens are all expressed in the robot base frame.

3.1 3D Scene Features

A pretrained VLM [6] encodes RGB observations and the language instruction ℓ into patch-level visual tokens f_v and language embeddings f_ℓ . Using depth and camera calibration, each visual token is associated with a 3D position in the robot base frame, producing lifted scene tokens f_v^{lift} . Proprioceptive features $\mathbf{p}$ are represented as learnable tokens anchored at the current end-effector pose.

3.2 3D Scene Trajectory Denoiser

We introduce a *3D Scene Trajectory Denoiser* that predicts future point motion in the robot base frame. Intermediate decoder states provide the latent motion representation used by the downstream action denoiser. Given the front-camera RGB-D observation (I, D) and camera calibration, we sample a regular grid of image points and lift them into 3D point anchors $\{q_i^{(0)}\}_{i=1}^N$ in the robot base frame. Our motion decoder predicts future 3D point trajectories as per-step 3D displacement increments:

$$\mathcal{F} = \{\Delta q_i^{(t)}\}_{i=1, t=1}^{N, H} \in \mathbb{R}^{N \times H \times 3}, \quad (1)$$

where $\Delta q_i^{(t)}$ denotes the predicted displacement increment of point anchor i at time step t . Future 3D point positions are recovered through cumulative integration $q_i^{(t)} = q_i^{(0)} + \sum_{s=1}^t \Delta q_i^{(s)}$.

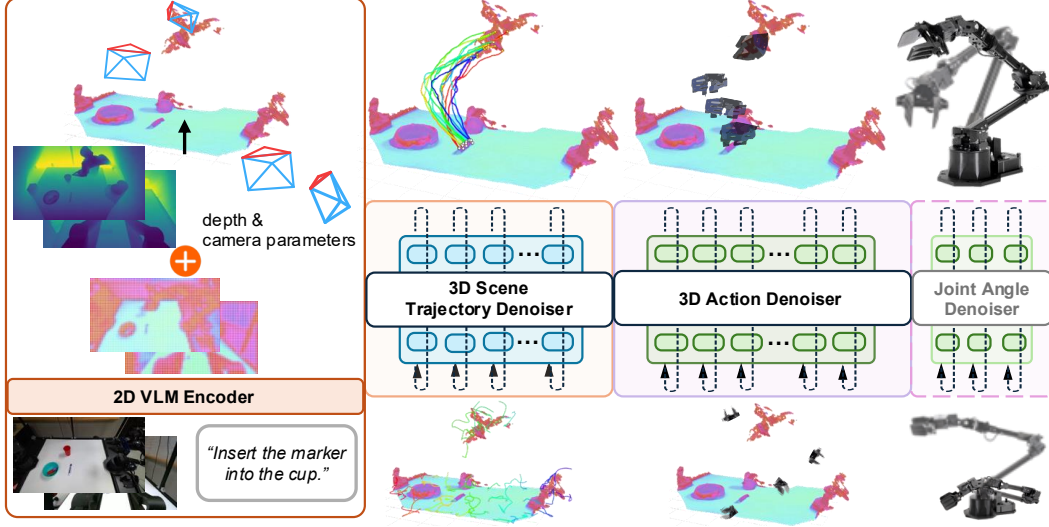


Figure 2: **GeomVLA architecture.** Left: A shared 2D VLM encodes multi-view RGB observations and language instructions. Depth and camera geometry lift image features into 3D scene tokens with positions in the robot base frame. Middle: The 3D Scene Trajectory Denoiser is trained to predict future trajectories for front-view 3D anchors. During policy inference, one velocity-field evaluation provides intermediate motion features that are fused with the 3D scene tokens. Right: The 3D Action Denoiser predicts end-effector trajectory chunks conditioned on the fused scene-motion representation, language features, and robot proprioception. Optionally, the Joint Angle Denoiser uses the scene-motion representation and end-effector action features to predict joint-angle commands.

Using the VLM features from §3.1, we bilinearly sample visual descriptors at the query locations and combine them with projected 3D coordinates. Local geometry-aware attention then produces task-conditioned 3D scene tokens $\{c_i\}_{i=1}^N$.

Conditioned on $\mathcal{C} = (\{c_i\}, V_g, E)$, where V_g and E denote global visual and language tokens, a lightweight flow-matching decoder jointly predicts displacement trajectories for all anchors and future time steps. The decoder alternates serialized spatial attention over 3D anchors with temporal attention over future steps. Full encoder, decoder, and loss-weight details are provided in the appendix.

The 3D Scene Trajectory Denoiser is trained using SpatialTrackerV2 pseudo-labels extracted from benchmark demonstration videos. We supervise future displacement trajectories in the robot base frame using a motion-weighted rectified-flow objective:

$$\mathcal{L}_{\text{traj}} = \mathbb{E}_{\tau, \mathcal{F}_0} \left[\sum_{i=1}^N \sum_{t=1}^H w_{i,t} \|u_{\theta}(\mathcal{F}_{\tau}, \tau, \mathcal{C})_{i,t} - u_{i,t}^*\|_2^2 \right], \quad (2)$$

where τ is the flow-matching time, $\mathcal{F}_0$ a Gaussian noise trajectory, $\mathcal{F}_{\tau}$ the interpolated trajectory, $\mathcal{C}$ the visual-language context, u_{θ} the predicted velocity field, $u_{i,t}^*$ the target flow at anchor i and step t , and $w_{i,t}$ upweights motion-relevant points while retaining static structure.

3.3 3D Action Denoiser

The 3D action denoiser predicts end-effector trajectories from lifted scene tokens f_v^{lift} , language tokens f_{ℓ} , proprioception $\mathbf{p}$, and the latent motion features described in §3.2.

At the initial flow state $\tau = 0$, we run the trajectory denoiser once on Gaussian noise, without integrating a complete trajectory. Its intermediate decoder tensor contains one feature for each of the $N = 400$ anchors and H future time steps. A learned temporal-attention query pools the H features for each anchor into one motion token z_i^{m} . A single geometry-aware cross-attention layer

then fuses these motion tokens into the scene tokens, with the residual contribution scaled by a zero-initialized learned scalar, yielding motion-aware scene tokens f_v^{mot} before action generation.

The 3D action denoiser represents noisy end-effector trajectories as spatial action tokens with explicit 3D positions. A stack of geometry-aware transformer blocks alternates cross-attention to scene and language tokens with self-attention over action trajectories to produce the action feature f_a :

$$f_a \leftarrow \text{CrossAttn}_{3D}(f_a, [f_v^{\text{mot}}; f_\ell]), \quad (3)$$

$$f_a \leftarrow \text{SelfAttn}_{3D}(f_a), \quad (4)$$

$$f_a \leftarrow \text{adaLN-FFN}(f_a; \lambda, \mathbf{p}), \quad (5)$$

where λ is the flow time. All attention layers use 3D rotary positional embeddings derived from token positions.

The 3D action denoiser predicts rectified-flow velocities for end-effector translation and rotation, together with binary gripper-state predictions. We optimize

$$\mathcal{L}_\pi = \mathbb{E}_{\lambda, \mathcal{A}_0} \left[\|v_\theta(\mathcal{A}_\lambda, \lambda, f_a) - v^*\|_2^2 + \text{BCE}(f_\theta^{\text{open}}, a^{\text{open}}) \right], \quad (6)$$

where $\mathcal{A}_0$ is a Gaussian noise action, $\mathcal{A}_\lambda$ the interpolated action, and v^* the target velocity. BCE supervises the gripper-open prediction f_θ^{open} against ground-truth a^{open} .

3D reasoning for joint-space control. GeomVLA’s 3D perception-action pathway can also serve as an intermediate geometric reasoning process for joint-space control. Instead of executing the predicted end-effector actions directly, we condition a downstream joint-angle action denoiser on f_ℓ , f_v^{mot} , and the end-effector action feature after one velocity-field evaluation. The joint-angle denoiser is trained with the same rectified-flow objective as Eq. (6).

3.4 Training

Training proceeds in two stages. First, we train the 3D Scene Trajectory Denoiser from Spatial-TrackerV2 point-track pseudo-labels while keeping the VLM frozen. Second, we jointly fine-tune the action denoiser, pretrained trajectory denoiser, and VLM using only the action loss in Eq. (6); the trajectory pseudo-label loss is not retained in this stage. GeomVLA uses the public Florence-2 initialization but no robot-action pretraining and is trained only on the benchmark demonstrations.

Implementation details. To obtain pseudo-labels for the trajectory predictor, we use Spatial-TrackerV2 [55] to estimate 3D point trajectories from videos. The predictor uses a 20×20 query grid of 3D anchors. We temporally downsample each video by a factor of three, and the predictor estimates a horizon of 15 future time steps in the downsampled sequence, spanning 45 frames at the original frame rate. The trajectories are represented as per-step 3D displacement increments in the robot base frame.

4 Experiments

We evaluate GeomVLA through experiments designed to isolate the role of explicit 3D future-motion reasoning in vision-language-action control: (i) whether latent 3D scene-motion prediction improves an otherwise identical 3D action-only policy, (ii) whether a shared 3D representation is beneficial, (iii) how the method compares with recent 2D and 3D VLAs and motion-guided policies, and (iv) real-world performance across varied objects, distractors, and layouts.

4.1 Simulation Experiments

We evaluate GeomVLA on CALVIN [27], LIBERO [28], and RoboTwin2.0 [29], spanning long-horizon manipulation, compositional generalization, and single-/dual-arm control. The policy outputs binary gripper-state predictions and end-effector deltas for translation and rotation. For

Table 1: **Comparison with prior methods on simulation benchmarks.** GeomVLA-JA denotes the joint-angle action variant conditioned on 3D scene-motion features and intermediate end-effector action features; 2D-JA is a 2D VLA baseline using the same VLM, joint-angle action denoiser, and training data as GeomVLA-JA; GeomVLA w/o motion removes the 3D Scene Trajectory Denoiser while retaining the 3D action policy. *Pretrained* indicates large-scale robot-action pretraining. For RoboTwin2.0, (a)–(e) are a five-task evaluation subset selected following SimpleVLA-RL [56], each reported as the mean success rate across the Easy and Hard settings: (a) `beat_block_hammer`, (b) `pick_dual_bottles`, (c) `move_can_pot`, (d) `handover_mic`, and (e) `stack_bowls_two`. Tasks (a)–(b), (c)–(d), and (e) are short-, medium-, and long-horizon tasks, with approximately 100, 200, and 300 steps, respectively. We fine-tune $\pi_{0.5}$ and train full GeomVLA on all 50 tasks; GeomVLA w/o motion, 2D-JA, and GeomVLA-JA are trained only on the five selected tasks. The five-task comparison therefore uses different training sets and is not a fully controlled ablation.

Method	Pretrained	Params	CALVIN	LIBERO					RoboTwin2.0					
			ABC-D	Spatial	Object	Goal	Long	Avg.	a	b	c	d	e	Avg.
3D VLA														
3D-VLA	Yes	–	0.707	–	–	–	–	–	–	–	–	–	–	–
SpatialVLA	Yes	4B	–	88.2	89.9	78.6	55.5	78.1	–	–	–	–	–	–
4D-VLA	Yes	4B+	–	88.9	95.2	90.0	79.1	88.6	–	–	–	–	–	–
PoseVLA	Yes	3B+	–	96.5	98.0	97.1	92.4	96.0	93.5	87.0	58.0	<u>93.0</u>	96.5	85.6
<i>Video/Motion-Guided</i>														
TraceVLA	Yes	7B	–	84.6	85.2	75.1	54.1	74.8	–	–	–	–	–	–
mimic-video	Yes	2B+	–	94.2	96.8	90.6	–	93.9	–	–	–	–	–	–
WorldVLA	No	7B	–	87.6	96.2	83.4	60.0	81.8	–	–	–	–	–	–
UniVLA	Yes	7B	3.801	96.5	96.8	95.6	92.0	95.2	–	–	–	–	–	–
CoT-VLA	Yes	7B	–	87.5	91.6	87.6	69.0	81.1	–	–	–	–	–	–
FlowVLA	Yes	8.5B	–	93.2	95.0	91.6	72.6	88.1	–	–	–	–	–	–
LaMP	Yes	5B	–	99.4	99.8	97.4	96.7	98.3	–	–	–	–	–	–
RoboFlow4D	Yes	0.76B+	–	90.2	97.0	88.4	75.2	87.7	–	–	–	–	–	–
<i>Others</i>														
$\pi_{0.5}$	Yes	3B+	3.964	98.8	98.2	<u>98.0</u>	92.4	96.8	82.0	94.0	90.0	47.0	<u>93.0</u>	81.2
OpenVLA-OFT	Yes	7B	4.279	97.6	98.4	97.9	94.5	97.1	28.1	29.7	28.1	45.3	40.6	34.4
FLOWER	Yes	0.95B	4.480	97.2	<u>99.3</u>	96.9	94.5	97.0	–	–	–	–	–	–
SimpleVLA-RL	Yes	7B	–	99.4	99.1	99.2	98.5	99.1	87.5	68.3	61.2	89.2	75.8	76.4
X-VLA	Yes	0.9B	4.430	98.2	98.6	97.8	<u>97.6</u>	98.1	48.0	28.5	47.0	69.0	46.5	47.8
2D-JA	No	1B	4.195	–	–	–	–	–	39.0	77.0	88.0	74.0	53.0	66.2
GeomVLA-JA	No	1.3B	<u>4.508</u>	–	–	–	–	–	71.0	75.0	<u>91.0</u>	95.0	76.0	81.6
GeomVLA w/o motion	No	1.2B	<u>4.508</u>	94.8	99.2	88.2	91.8	93.5	45.0	<u>96.0</u>	83.0	83.0	43.0	70.0
GeomVLA	No	1.2B	4.624	98.6	99.8	97.6	<u>97.6</u>	<u>98.4</u>	<u>89.0</u>	98.0	99.0	55.0	83.0	<u>84.8</u>

RoboTwin2.0 and the real-world tasks, every predicted pose in a chunk is a current-state-relative delta from the observed pose. For CALVIN and LIBERO, we use chained per-step deltas, with each pose defined relative to the preceding pose in the chunk.

Results. Full results are in Table 1, with qualitative rollouts shown in Figure 3. GeomVLA achieves state-of-the-art performance on CALVIN and competitive performance on LIBERO and RoboTwin2.0 against (i) 2D VLAs [3, 57, 7, 56, 8], (ii) 3D policies and VLAs [14, 15, 16, 58], and (iii) motion-guided VLAs [22, 59, 20, 41, 60, 21, 46, 25]. On CALVIN, it improves over FLOWER (4.480 $\rightarrow$ 4.624) and X-VLA (4.430 $\rightarrow$ 4.624) without additional robot-action pretraining. On LIBERO, it attains the best average performance among methods without reinforcement-learning fine-tuning. Improvements are most pronounced in long-horizon and contact-rich tasks requiring precise spatial reasoning and delayed interaction effects. Across all 50 RoboTwin2.0 tasks, GeomVLA obtains 78.6%/76.2% success under the Easy/Hard settings, surpassing 75.9%/75.7% for a fully fine-tuned $\pi_{0.5}$ (Appendix Table 5). These results support the benefit of maintaining geometric consistency among scene representation, future reasoning, and action generation.

4.2 Ablation Studies

We analyze two questions: (i) the effect of future-motion reasoning, and (ii) the role of shared 3D representation. Table 2 summarizes the ablative variants of our model and their CALVIN ABC-D performance. We draw the following conclusions:

Future-motion reasoning helps both 2D (4.411 $\rightarrow$ 4.437) and 3D policies (4.508 $\rightarrow$ 4.624), suggesting that predicting scene evolution provides useful temporal structure beyond action imitation.

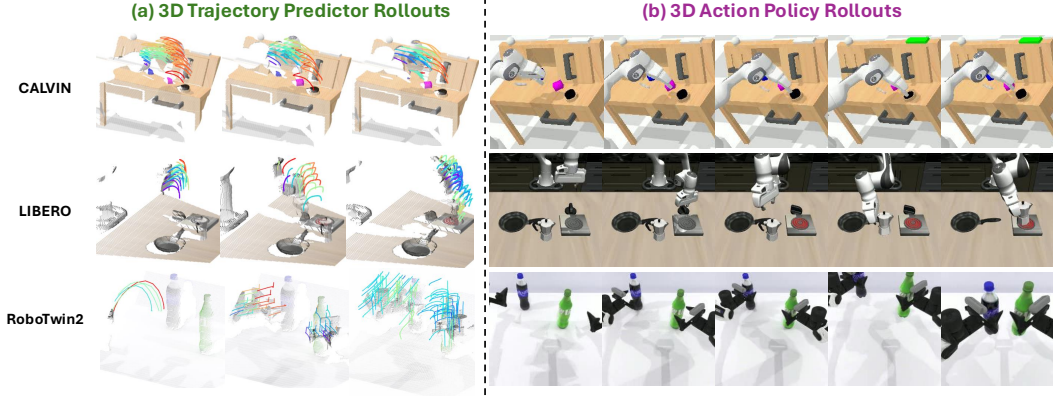


Figure 3: **Simulation examples from CALVIN, LIBERO, and RoboTwin2.0.** Left: predicted 3D point trajectories from the Scene Trajectory Denoiser. Right: corresponding closed-loop policy rollouts.

Table 2: **CALVIN ABC-D ablations.** The metric is the mean number of consecutive tasks completed in each five-instruction sequence. The full model performs best among the evaluated variants.

Variant	Description	Perf.
GeomVLA w/o motion	Removes the 3D Scene Trajectory Denoiser.	4.508
2DGeomVLA w/o motion	Uses a 2D image-based policy without future-motion reasoning.	4.411
2DGeomVLA	Uses 2D future-motion reasoning with a 2D action denoiser.	4.437
2DGeomVLA LaMP-style	Uses pixel (u, v) coordinates and metric depth for scene motion.	4.462
GeomVLA w/ 2D motion	Uses image-space future motion with the 3D action denoiser.	4.085
GeomVLA w/ denoised trace	Conditions action denoising on the fully denoised 3D trajectory.	4.047
GeomVLA w/ denoised latent	Conditions action denoising on the fully denoised motion latent.	4.334
GeomVLA w/ partial denoising	Reads the motion latent at $\tau = 0.1$ instead of pure noise.	4.484
GeomVLA encoder-only	Uses only the scene trajectory encoder, without velocity prediction.	4.443
GeomVLA w/ arbitrary frame	Expresses scene, motion, and action in one arbitrary rigid frame.	4.596
GeomVLA	Unifies scene, future motion, and action in a shared 3D frame.	4.624

Explicit 3D geometry in VLAs helps. Replacing the 2D action denoiser with the corresponding 3D action denoiser improves performance from 4.411 to 4.508, showing the benefit of representing scene features and robot actions directly in metric 3D coordinates.

Scene motion representation matters. The full model conditions the action denoiser on the intermediate decoder state from one velocity-field evaluation at the initial noisy flow state. We find that weaker alternatives all underperform: using fully denoised trajectories (4.047), fully denoised latents (4.334), partially denoised latents at $\tau = 0.1$ (4.484), or only the trajectory encoder (4.443). These results suggest that the action denoiser benefits most from early-stage motion latents that capture how the scene should evolve, rather than from finalized trajectory estimates or encoder features alone.

Geometric consistency matters. When future-motion reasoning is performed in image space but action generation remains in 3D, performance drops to 4.085, below the fully 2D model. The LaMP-style variant, which reasons over image-plane (u, v) coordinates plus metric depth and removes 3D RoPE, reaches 4.462, suggesting that mixing image coordinates with metric depth provides a less coherent geometry for spatial reasoning. In contrast, applying the same arbitrary rigid transform to scene, motion, and action coordinates preserves performance (4.596 versus 4.624). Together, these ablations indicate that the benefit is associated with a consistent metric geometry, rather than with the robot base frame specifically. Holding the image fixed while replacing the instruction increases trajectory endpoint error by 3.40 cm on CALVIN, 5.44 cm on LIBERO, and 1.64 cm on RoboTwin2.0, showing that the predicted motion depends on language conditioning (Appendix Table 7). Additional ablation definitions and stability analyses are provided in the appendix.

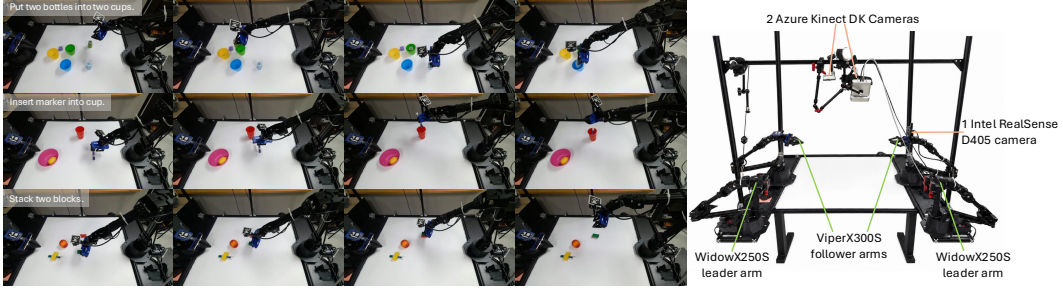


Figure 4: **Real-world evaluation setup and tasks.** We evaluate GeomVLA on an ALOHA2 platform on single-arm tabletop tasks. The left panel shows representative rollouts on three real-world tasks: placing two bottles into two cups, inserting a marker into a cup, and stacking two blocks.

4.3 Real-World Experiments

Setup. We evaluate GeomVLA on an ALOHA2 robot platform using the right ViperX300S follower arm (Figure 4). Demonstrations are collected through teleoperation with the corresponding WidowX250S leader arm. Visual observations are captured by two elevated Microsoft Azure Kinect DK cameras mounted at the front and back of the workspace, together with a wrist-mounted Intel RealSense D405 camera on the right follower arm. Our real-world evaluation focuses on single-arm tabletop manipulation tasks.

Tasks. We evaluate eight real-world tabletop manipulation tasks spanning precision insertion, stacking, object placement, and multi-stage interaction: inserting a marker into a cup, stacking two blocks, placing two bottles into matching cups, standing a bottle upright, transferring a marker between cups, uncapping a marker, ranking three bottles by color, and arranging three bottles into a triangle. The first three tasks use 50 demonstration rollouts each, and the remaining five use 20 rollouts each. The appendix defines all tasks and shows additional qualitative rollouts. For every variant, we train one multi-task policy on the same demonstrations from all eight tasks. GeomVLA and GeomVLA w/o motion predict end-effector poses and are trained from the collected demonstrations only. The $\pi_{0.5}$ baseline predicts joint-angle actions and is fine-tuned with LoRA from its public base checkpoint. Section 4.4 separately compares 2D-JA and GeomVLA-JA under the same joint-angle output parameterization.

Results. Results are summarized in Table 3. Across the eight tasks, GeomVLA achieves the highest mean success rate (62.5%), outperforms GeomVLA without motion on every task, and exceeds $\pi_{0.5}$ on seven of eight tasks. Rank by color remains challenging. We hypothesize that its visually similar intermediate states to Place in triangle create cross-task interference, causing hesitation over the long horizon.

Qualitative rollouts show examples in which GeomVLA continues successfully after transient execution errors, partial occlusion, or small object-pose inaccuracies (Appendix Figure 5).

4.4 3D Reasoning for Joint-Angle Robot Control

GeomVLA performs geometric reasoning over scene features, future motion, and end-effector actions in a shared 3D frame, but the executed action need not use a Cartesian parameterization. GeomVLA-JA adds a joint-angle denoiser conditioned on f_ℓ , f_v^{mot} , and the end-effector action features obtained after one velocity-field evaluation. These intermediate representations guide the prediction of executable joint-angle commands and allow us to compare 2D and 3D reasoning under a matched output action space. Joint-angle output avoids executing the predicted Cartesian trajectory directly.

Table 3: **Real-world success rates on ALOHA2.** Each entry reports the number of successful trials among 20 evaluations. GeomVLA-JA denotes the joint-angle action variant conditioned on 3D scene-motion features and intermediate end-effector action features; 2D-JA uses the same VLM, joint-angle denoiser, and training data but performs perception and motion reasoning in 2D; GeomVLA w/o motion removes the 3D Scene Trajectory Denoiser; GeomVLA is the full Cartesian-action model. [†] denotes the three tasks with 50 demonstration rollouts each; the remaining five tasks use 20 demonstration rollouts.

Task	$\pi_{0.5}$	2D-JA	GeomVLA-JA	GeomVLA w/o motion	GeomVLA
Insert marker [†]	9/20	3/20	13/20	3/20	5/20
Stack blocks [†]	2/20	9/20	17/20	15/20	19/20
Place bottles [†]	10/20	16/20	16/20	14/20	17/20
Stand bottle	7/20	6/20	13/20	13/20	17/20
Transfer marker	2/20	0/20	13/20	10/20	12/20
Uncap marker	15/20	12/20	18/20	15/20	20/20
Rank by color	0/20	0/20	1/20	0/20	2/20
Place in triangle	2/20	0/20	4/20	1/20	8/20
Average	29.4%	28.8%	<u>59.4%</u>	44.4%	62.5%

Results. As shown in Tables 1 and 3, GeomVLA-JA and 2D-JA use the same joint-angle action parameterization and are trained on the same benchmark data. GeomVLA-JA outperforms 2D-JA on CALVIN (4.508 versus 4.195), the five-task RoboTwin2.0 subset (81.6% versus 66.2%), and the real-world evaluation (59.4% versus 28.8%). This matched comparison supports the benefit of the 3D scene-motion and end-effector action representations rather than joint-angle prediction alone. GeomVLA-JA also approaches the full Cartesian-action model on average and performs best on marker insertion (13/20, compared with 5/20 for GeomVLA and 9/20 for $\pi_{0.5}$). Thus, the benefit of GeomVLA’s geometric representation persists under both Cartesian and joint-angle output parameterizations.

5 Limitations

GeomVLA demonstrates strong performance across simulation and real-world manipulation benchmarks, but several limitations remain. First, it relies on accurate camera calibration; depth measurement errors or camera-extrinsic calibration errors can degrade the quality of geometric representations, particularly in real-world settings. Second, unlike recent large-scale VLA systems, GeomVLA is trained only on benchmark-scale robot demonstrations without large-scale robot pretraining or cross-embodiment data. Scaling to larger and more diverse datasets could further improve semantic generalization, robustness, and long-horizon reasoning. Finally, GeomVLA performs reasoning through latent geometric trajectory representations rather than explicit language-based reasoning. Combining geometric reasoning with language-conditioned planning or hierarchical task decomposition is a promising direction for long-horizon manipulation and failure recovery.

6 Conclusion

We presented GeomVLA, a 3D motion-aware vision-language-action framework that unifies semantic perception, latent future-motion reasoning, and action generation within a shared metric 3D coordinate frame. Our approach introduces a *3D Scene Trajectory Denoiser* that predicts future scene motion directly in 3D space and integrates intermediate motion features into a flow-based 3D action denoiser through geometry-aware attention. Experiments across CALVIN, LIBERO, RoboTwin2.0, and real-world manipulation benchmarks show that maintaining geometric consistency between scene representation, future reasoning, and robot actions improves manipulation performance. These results motivate VLA architectures that couple semantic representations with physically grounded spatial and temporal reasoning.

References

- [1] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, T.-W. E. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. Rt-2: Vision-language-action models transfer web knowledge to robotic control, 2023. URL <https://arxiv.org/abs/2307.15818>.
- [2] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0 : A vision-language-action flow model for general robot control, 2026. URL <https://arxiv.org/abs/2410.24164>.
- [3] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025. URL <https://arxiv.org/abs/2504.16054>.
- [4] L. Beyer, A. Steiner, A. S. Pinto, A. Kolesnikov, X. Wang, D. Salz, M. Neumann, I. Al-abdulmohsin, M. Tschannen, E. Bugliarello, T. Unterthiner, D. Keysers, S. Koppula, F. Liu, A. Grycner, A. Gritsenko, N. Houlsby, M. Kumar, K. Rong, J. Eisenschlos, R. Kabra, M. Bauer, M. Bošnjak, X. Chen, M. Minderer, P. Voigtlaender, I. Bica, I. Balazevic, J. Puigcerver, P. Papalampidi, O. Henaff, X. Xiong, R. Soricut, J. Harmsen, and X. Zhai. Paligemma: A versatile 3b vlm for transfer, 2024. URL <https://arxiv.org/abs/2407.07726>.
- [5] Qwen, :, A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Tang, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- [6] B. Xiao, H. Wu, W. Xu, X. Dai, H. Hu, Y. Lu, M. Zeng, C. Liu, and L. Yuan. Florence-2: Advancing a unified representation for a variety of vision tasks, 2023. URL <https://arxiv.org/abs/2311.06242>.
- [7] M. Reuss, H. Zhou, M. Rühle, Ömer Erdiñç Yağmurlu, F. Otto, and R. Lioutikov. Flower: Democratizing generalist robot policies with efficient vision-language-action flow policies, 2025. URL <https://arxiv.org/abs/2509.04996>.
- [8] J. Zheng, J. Li, Z. Wang, D. Liu, X. Kang, Y. Feng, Y. Zheng, J. Zou, Y. Chen, J. Zeng, Y.-Q. Zhang, J. Pang, J. Liu, T. Wang, and X. Zhan. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model, 2025. URL <https://arxiv.org/abs/2510.10274>.
- [9] T. Gervet, Z. Xian, N. Gkanatsios, and K. Fragkiadaki. Act3d: 3d feature field transformers for multi-task robotic manipulation. *CoRL*, 2023.
- [10] M. Shridhar, L. Manuelli, and D. Fox. Perceiver-actor: A multi-task transformer for robotic manipulation. In *Conference on Robot Learning*, pages 785–799. PMLR, 2023.
- [11] T.-W. Ke, N. Gkanatsios, and K. Fragkiadaki. 3d diffuser actor: Policy diffusion with 3d scene representations. *arXiv preprint arXiv:2402.10885*, 2024.

- [12] A. Goyal, J. Xu, Y. Guo, V. Blukis, Y.-W. Chao, and D. Fox. Rvt: Robotic view transformer for 3d object manipulation. *arXiv preprint arXiv:2306.14896*, 2023.
- [13] R. Yang, G. Chen, C. Wen, and Y. Gao. Fp3: A 3d foundation policy for robotic manipulation, 2025.
- [14] H. Zhen, X. Qiu, P. Chen, J. Yang, X. Yan, Y. Du, Y. Hong, and C. Gan. 3d-vla: A 3d vision-language-action generative world model, 2024. URL <https://arxiv.org/abs/2403.09631>.
- [15] D. Qu, H. Song, Q. Chen, Y. Yao, X. Ye, Y. Ding, Z. Wang, J. Gu, B. Zhao, D. Wang, and X. Li. Spatialvla: Exploring spatial representations for visual-language-action model, 2025. URL <https://arxiv.org/abs/2501.15830>.
- [16] J. Zhang, Y. Chen, Y. Xu, Z. Huang, Y. Zhou, Y.-J. Yuan, X. Cai, G. Huang, X. Quan, H. Xu, and L. Zhang. 4d-vla: Spatiotemporal vision-language-action pretraining with cross-scene calibration, 2025. URL <https://arxiv.org/abs/2506.22242>.
- [17] Y. Jia, J. Liu, S. Chen, C. Gu, Z. Wang, L. Luo, L. Lee, P. Wang, Z. Wang, R. Zhang, and S. Zhang. Lift3d foundation policy: Lifting 2d large-scale pretrained models for robust 3d robotic manipulation, 2024. URL <https://arxiv.org/abs/2411.18623>.
- [18] C. Li, J. Wen, Y. Peng, Y. Peng, F. Feng, and Y. Zhu. Pointvla: Injecting the 3d world into vision-language-action models, 2025. URL <https://arxiv.org/abs/2503.07511>.
- [19] L. Sun, B. Xie, Y. Liu, H. Shi, T. Wang, and J. Cao. Geovla: Empowering 3d representations in vision-language-action models. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2026.
- [20] J. Cen, C. Yu, H. Yuan, Y. Jiang, S. Huang, J. Guo, X. Li, Y. Song, H. Luo, F. Wang, D. Zhao, and H. Chen. Worldvla: Towards autoregressive action world model, 2025. URL <https://arxiv.org/abs/2506.21539>.
- [21] Z. Zhong, H. Yan, J. Li, X. Liu, X. Gong, T. Zhang, W. Song, J. Chen, X. Zheng, H. Wang, and H. Li. Flowvla: Visual chain of thought-based motion reasoning for vision-language-action models, 2025. URL <https://arxiv.org/abs/2508.18269>.
- [22] R. Zheng, Y. Liang, S. Huang, J. Gao, H. D. III, A. Kolobov, F. Huang, and J. Yang. Tracevla: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies, 2025. URL <https://arxiv.org/abs/2412.10345>.
- [23] J. Gu, S. Kirmani, P. Wohlhart, Y. Lu, M. G. Arenas, K. Rao, W. Yu, C. Fu, K. Gopalakrishnan, Z. Xu, P. Sundaresan, P. Xu, H. Su, K. Hausman, C. Finn, Q. Vuong, and T. Xiao. Rt-trajectory: Robotic task generalization via hindsight trajectory sketches, 2023. URL <https://arxiv.org/abs/2311.01977>.
- [24] S. Lee, Y. Jung, I. Chun, Y.-C. Lee, Z. Cai, H. Huang, A. Talreja, T. D. Dao, Y. Liang, J.-B. Huang, and F. Huang. Tracegen: World modeling in 3d trace space enables learning from cross-embodiment videos, 2025. URL <https://arxiv.org/abs/2511.21690>.
- [25] S. Lin, J. Chen, H. Xu, Z. Li, G. Wang, Y. Jing, S. Xu, R. Zhao, B. Sheil, L.-P. Chau, and G. Liu. Roboflow4d: A lightweight flow world model toward real-time flow-guided robotic manipulation, 2026. URL <https://arxiv.org/abs/2605.17522>.
- [26] H. Zhi, P. Chen, S. Zhou, Y. Dong, Q. Wu, L. Han, and M. Tan. 3dflowaction: Learning cross-embodiment manipulation from 3d flow world model, 2025. URL <https://arxiv.org/abs/2506.06199>.

- [27] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard. Calvin: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters (RA-L)*, 7(3):7327–7334, 2022.
- [28] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36, 2024.
- [29] T. Chen, Z. Chen, B. Chen, Z. Cai, Y. Liu, Z. Li, Q. Liang, X. Lin, Y. Ge, Z. Gu, W. Deng, Y. Guo, T. Nian, X. Xie, Q. Chen, K. Su, T. Xu, G. Liu, M. Hu, H. ang Gao, K. Wang, Z. Liang, Y. Qin, X. Yang, P. Luo, and Y. Mu. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation, 2025. URL <https://arxiv.org/abs/2506.18088>.
- [30] Z. Xian, N. Gkanatsios, T. Gervet, T.-W. Ke, and K. Fragkiadaki. Chaineddiffuser: Unifying trajectory diffusion and keypose prediction for robotic manipulation. In *Conference on Robot Learning*, pages 2323–2339. PMLR, 2023.
- [31] E. Chisari, N. Heppert, M. Argus, T. Welschhold, T. Brox, and A. Valada. Learning robotic manipulation policies from point clouds with conditional flow matching. *arXiv preprint arXiv:2409.07343*, 2024.
- [32] I.-C. A. Liu, S. He, D. Seita, and G. Sukhatme. Voxact-b: Voxel-based acting and stabilizing policy for bimanual manipulation. *CoRL*, 2024.
- [33] M. Grotz, M. Shridhar, T. Asfour, and D. Fox. Peract2: A perceiver actor framework for bimanual manipulation tasks. *arXiv preprint arXiv:2407.00278*, 2024.
- [34] N. Gkanatsios, J. Xu, M. Bronars, A. Mousavian, T.-W. Ke, and K. Fragkiadaki. 3d flowmatch actor: Unified 3d policy for single- and dual-arm manipulation, 2025. URL <https://arxiv.org/abs/2508.11002>.
- [35] P. Li, Y. Chen, H. Wu, X. Ma, X. Wu, Y. Huang, L. Wang, T. Kong, and T. Tan. Bridgevla: Input-output alignment for efficient 3d manipulation learning with vision-language models, 2025. URL <https://arxiv.org/abs/2506.07961>.
- [36] T. Lin, G. Li, Y. Zhong, Y. Zou, Y. Du, J. Liu, E. Gu, and B. Zhao. Evo-0: Vision-language-action model with implicit spatial understanding, 2025. URL <https://arxiv.org/abs/2507.00416>.
- [37] S. Deng, M. Yan, Y. Zheng, J. Su, W. Zhang, X. Zhao, H. Cui, Z. Zhang, and H. Wang. Stereovla: Enhancing vision-language-action models with stereo vision, 2025. URL <https://arxiv.org/abs/2512.21970>.
- [38] X. Fan, S. Deng, X. Wu, Y. Lu, Z. Li, M. Yan, Y. Zhang, Z. Zhang, H. Wang, and H. Zhao. Any3d-vla: Enhancing vla robustness via diverse point clouds, 2026. URL <https://arxiv.org/abs/2602.00807>.
- [39] W. Li, J. Liu, L. Yixing, J. Tong, R. Shao, and L. Nie. Consisvla-4d: Advancing spatiotemporal consistency in efficient 3d-perception and 4d-reasoning for robotic manipulation, 2026. URL <https://arxiv.org/abs/2605.05126>.
- [40] Y. Du, M. Yang, B. Dai, H. Dai, O. Nachum, J. B. Tenenbaum, D. Schuurmans, and P. Abbeel. Learning universal policies via text-guided video generation, 2023. URL <https://arxiv.org/abs/2302.00111>.
- [41] S. Li, Y. Gao, D. Sadigh, and S. Song. Unified video action model, 2025. URL <https://arxiv.org/abs/2503.00200>.

- [42] H. Bi, H. Tan, S. Xie, Z. Wang, S. Huang, H. Liu, R. Zhao, Y. Feng, C. Xiang, Y. Rong, H. Zhao, H. Liu, Z. Su, L. Ma, H. Su, and J. Zhu. Motus: A unified latent action world model, 2025. URL <https://arxiv.org/abs/2512.13030>.
- [43] G. Team, B. Wang, B. Li, C. Ni, G. Huang, G. Zhao, H. Li, J. Li, J. Lv, J. Liu, L. Feng, M. Yu, P. Li, Q. Deng, T. Liu, X. Zhou, X. Chen, X. Wang, Y. Wang, Y. Li, Y. Nie, Y. Li, Y. Zhou, Y. Ye, Z. Liu, and Z. Zhu. Gigabrain-0.5m*: a vla that learns from world model-based reinforcement learning, 2026. URL <https://arxiv.org/abs/2602.12099>.
- [44] M. Xu, Z. Xu, Y. Xu, C. Chi, G. Wetzstein, M. Veloso, and S. Song. Flow as the cross-domain manipulation interface, 2024. URL <https://arxiv.org/abs/2407.15208>.
- [45] S. Noh, D. Nam, K. Kim, G. Lee, Y. Yu, R. Kang, and K. Lee. 3d flow diffusion policy: Visuomotor policy learning via generating flow in 3d space, 2025. URL <https://arxiv.org/abs/2509.18676>.
- [46] X. Wang, C. Wang, Y. Xu, M. Ye, F.-C. Zhang, J. Tian, X. Zhan, L. Zhu, C. Lu, and L. Yang. Lamp: Learning vision-language-action policies with 3d scene flow as latent motion prior, 2026. URL <https://arxiv.org/abs/2603.25399>.
- [47] V. Vosylius, Y. Seo, J. Uruç, and S. James. Render and diffuse: Aligning image and action spaces for diffusion-based behaviour cloning, 2024.
- [48] H. Bharadhwaj, R. Mottaghi, A. Gupta, and S. Tulsiani. Track2act: Predicting point tracks from internet videos enables generalizable robot manipulation, 2024. URL <https://arxiv.org/abs/2405.01527>.
- [49] C. Wen, X. Lin, J. So, K. Chen, Q. Dou, Y. Gao, and P. Abbeel. Any-point trajectory modeling for policy learning, 2024. URL <https://arxiv.org/abs/2401.00025>.
- [50] M. Zawalski, W. Chen, K. Pertsch, O. Mees, C. Finn, and S. Levine. Robotic control via embodied chain-of-thought reasoning, 2025. URL <https://arxiv.org/abs/2407.08693>.
- [51] Y. Li, Y. Deng, J. Zhang, J. Jang, M. Memmel, R. Yu, C. R. Garrett, F. Ramos, D. Fox, A. Li, A. Gupta, and A. Goyal. Hamster: Hierarchical action models for open-world robot manipulation, 2025. URL <https://arxiv.org/abs/2502.05485>.
- [52] C.-C. Hsu, B. Wen, J. Xu, Y. Narang, X. Wang, Y. Zhu, J. Biswas, and S. Birchfield. Spot: Se(3) pose trajectory diffusion for object-centric manipulation. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [53] J. Lee, J. Duan, H. Fang, Y. Deng, S. Liu, B. Li, B. Fang, J. Zhang, Y. R. Wang, S. Lee, W. Han, W. Pumacay, A. Wu, R. Hendrix, K. Farley, E. Vanderbilt, A. Farhadi, D. Fox, and R. Krishna. Molmoact: Action reasoning models that can reason in space, 2025. URL <https://arxiv.org/abs/2508.07917>.
- [54] Y. Li, Y. Chen, M. Zhou, H. Li, Z. Zhang, and D. Zhao. Qdepth-vla: Quantized depth prediction as auxiliary supervision for vision-language-action models, 2025. URL <https://arxiv.org/abs/2510.14836>.
- [55] Y. Xiao, J. Wang, N. Xue, N. Karaev, Y. Makarov, B. Kang, X. Zhu, H. Bao, Y. Shen, and X. Zhou. Spatialtrackerv2: 3d point tracking made easy, 2025. URL <https://arxiv.org/abs/2507.12462>.
- [56] H. Li, Y. Zuo, J. Yu, Y. Zhang, Z. Yang, K. Zhang, X. Zhu, Y. Zhang, T. Chen, G. Cui, D. Wang, D. Luo, Y. Fan, Y. Sun, J. Zeng, J. Pang, S. Zhang, Y. Wang, Y. Mu, B. Zhou, and N. Ding. Simplevla-rl: Scaling vla training via reinforcement learning, 2025. URL <https://arxiv.org/abs/2509.09674>.

- [57] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success, 2025. URL <https://arxiv.org/abs/2502.19645>.
- [58] H. Lin, H. Yu, J. Huang, H. Zhang, Y. Ling, P. Tan, X. Xue, and Y. Fu. Universal pose pretraining for generalizable vision-language-action policies, 2026. URL <https://arxiv.org/abs/2602.19710>.
- [59] J. Pai, L. Achenbach, V. Montesinos, B. Forrai, O. Mees, and E. Nava. mimic-video: Video-action models for generalizable robot control beyond vlas, 2025. URL <https://arxiv.org/abs/2512.15692>.
- [60] Q. Zhao, Y. Lu, M. J. Kim, Z. Fu, Z. Zhang, Y. Wu, Z. Li, Q. Ma, S. Han, C. Finn, A. Handa, M.-Y. Liu, D. Xiang, G. Wetzstein, and T.-Y. Lin. Cot-vla: Visual chain-of-thought reasoning for vision-language-action models, 2025. URL <https://arxiv.org/abs/2503.22020>.

Appendix

Model configurations and compute. Table 4 reports the model configuration used for each benchmark. All variants follow the same architectural design, while hidden dimensions, layer counts, action horizons, batch sizes, and optimization schedules are adapted to each benchmark. The second-stage joint training requires approximately 15 hours on four 48 GB NVIDIA L40S GPUs for CALVIN, 25 hours on eight 40 GB NVIDIA A100 GPUs for LIBERO, and 48 hours on eight 40 GB NVIDIA A100 GPUs for the five-task RoboTwin2.0 ablation setting.

Lifting 2D VLM features into 3D. We assign each patch token a depth value by bilinearly interpolating the four neighboring depth samples at the patch center. Camera intrinsics and extrinsics then map the patch center to a 3D position in the robot base frame.

3D scene trajectory encoder. The encoder binds VLM-derived semantics to local 3D geometry through a point-wise design. Each image query $\mathbf{u}_i = (u_i, v_i)$ is lifted to a base-frame point $\mathbf{q}_i$. We use centered coordinates $\tilde{\mathbf{q}}_i$ inside the trajectory encoder to remove global translation, while retaining the uncentered base-frame positions $\mathbf{q}_i$ for downstream spatial encoding:

$$\bar{\mathbf{u}}_i = [u_i, v_i, 1]^\top, \quad \mathbf{q}_i = R_{c \rightarrow b}(D(\mathbf{u}_i)K^{-1}\bar{\mathbf{u}}_i) + \mathbf{t}_{c \rightarrow b}, \quad \tilde{\mathbf{q}}_i = \mathbf{q}_i - \frac{1}{N} \sum_{j=1}^N \mathbf{q}_j. \quad (7)$$

Here K is the camera-intrinsic matrix, $D(\mathbf{u}_i)$ is the interpolated depth, and $R_{c \rightarrow b}$ and $\mathbf{t}_{c \rightarrow b}$ are the camera-to-base rotation and translation. A frozen Florence-2 VLM encodes the image into a patch feature map $V \in \mathbb{R}^{H_v \times W_v \times d_v}$ and the instruction into language tokens $E \in \mathbb{R}^{L \times d_t}$. For each query we bilinearly sample V to obtain a visual descriptor x_i and form an initial point token that fuses geometry, image location, and semantics:

$$h_i^{(0)} = \text{MLP}([\tilde{\mathbf{q}}_i; \mathbf{u}_i; x_i]) \in \mathbb{R}^D. \quad (8)$$

We build a k -nearest-neighbor graph $\mathcal{N}_k(i)$ once over $\{\tilde{\mathbf{q}}_i\}$ and refine tokens through L_{enc} local geometry-aware attention layers, where each query attends to its neighbors with attention logits augmented by a learned relative-position bias $b(\tilde{\mathbf{q}}_j - \tilde{\mathbf{q}}_i)$:

$$h_i^{(\ell+1)} = h_i^{(\ell)} + \sum_{j \in \mathcal{N}_k(i)} \alpha_{ij}^{(\ell)} W_v^{(\ell)} h_j^{(\ell)}, \quad \alpha_{ij}^{(\ell)} \propto \exp\left(\frac{(W_q^{(\ell)} h_i^{(\ell)})^\top (W_k^{(\ell)} h_j^{(\ell)})}{\sqrt{d_h}} + b(\tilde{\mathbf{q}}_j - \tilde{\mathbf{q}}_i)\right). \quad (9)$$

Here d_h is the per-head key dimension. The encoder outputs per-point context features $\{c_i\}_{i=1}^N$ that couple task-relevant semantics with local 3D structure. The decoder also receives the VLM’s pooled front-view visual tokens V_g and projected language tokens E as global multimodal conditioning, $\mathcal{C} = (\{c_i\}, V_g, E)$.

3D scene trajectory decoder. The decoder predicts $\mathcal{F}$ by conditional flow matching. Let $F \in \mathbb{R}^{N \times H \times 3}$ denote the normalized pseudo-ground-truth displacement tensor extracted by SpatialTrackerV2. Displacements are mapped into the normalized training coordinates before flow matching and mapped back with the inverse transformation at inference. We sample Gaussian noise $F_0 \sim \mathcal{N}(0, I)$ and a flow time $\tau \sim \mathcal{U}(0, 1)$, and define the linear interpolant and the velocity target

$$F_\tau = (1 - \tau) F_0 + \tau F, \quad u^* = F - F_0. \quad (10)$$

A velocity network $u_\theta(F_\tau, \tau, \mathcal{C})$ is trained to regress u^* , where $\mathcal{C} = (\{c_i\}, V_g, E)$ denotes the multimodal context.

The decoder input at point i and step t fuses the noisy displacement, the per-point context, a learned future-time-step embedding e_t and a sinusoidal flow-time embedding $\phi(\tau)$,

$$Z_{i,t}^{(0)} = W_F F_{\tau,i,t} + c_i + e_t + \phi(\tau). \quad (11)$$

Each decoder block factorizes attention along the spatial and temporal axes of the trajectory field. Spatial attention operates over 3D anchors at each future step. Rather than re-using the encoder’s kNN graph, we adopt a serialized window attention along a 3D space-filling curve, which scales to the full $N \times H$ token set with hardware-friendly fixed-size windows. Concretely, for each future step t we normalize the anchors to the unit cube, $\hat{q}_i = (\tilde{q}_i - \min_j \tilde{q}_j) / (\max_j \tilde{q}_j - \min_j \tilde{q}_j)$, quantize them to an integer grid at bit-depth d_q , and assign each anchor a 3D Morton (Z-order) code $m(\hat{q}_i) \in \mathbb{N}$ that yields a permutation π sorting the N points along the curve. Self-attention is then applied within non-overlapping windows of P consecutive points in the order π , and the result is restored to the original point order by the inverse permutation π^{-1} :

$$\text{SerSpatial}(Z)_{\pi(i),t} = \text{Attn}(Z_{\pi(i),t}; \{Z_{\pi(j),t} : j \in \mathcal{W}_P(i)\}), \quad \mathcal{W}_P(i) = \{j : \lfloor j/P \rfloor = \lfloor i/P \rfloor\}. \quad (12)$$

To enlarge the effective receptive field while keeping per-window cost $\mathcal{O}(P^2)$, we alternate between canonical (x, y, z) and axis-permuted (y, x, z) Morton orderings and apply a Swin-style half-window shift on alternating layers; a learned linear projection of the normalized coordinates is added to each token before the query, key, and value projections so that serialized attention remains explicitly geometry-aware.

For every point i , a temporal self-attention $\text{Temporal}(\cdot)$ couples its H future steps. The resulting tokens then attend to the conditioning sequence $[V_g; E]$ through a cross-attention block $\text{Cross}(\cdot, [V_g; E])$, followed by a position-wise feed-forward $\text{FFN}(\cdot)$. With pre-LayerNorm and residual connections, one block computes

$$Z \xrightarrow{\text{SerSpatial}} Z \xrightarrow{\text{Temporal}} Z \xrightarrow{\text{Cross}([V_g; E])} Z \xrightarrow{\text{FFN}} Z. \quad (13)$$

A linear head reads off the predicted velocity $u_\theta(F_\tau, \tau, \mathcal{C}) \in \mathbb{R}^{N \times H \times 3}$. We optimize the motion-weighted flow-matching objective

$$\mathcal{L} = \mathbb{E}_{\tau, F_0} \left[\sum_{i,t} w_{i,t} \|u_\theta(F_\tau, \tau, \mathcal{C})_{i,t} - u_{i,t}^*\|_2^2 \right], \quad (14)$$

where $w_{i,t} = m_{i,t} \cdot \max(\sigma((\|\Delta q_{i,t}\| - \tau_w) \kappa / \tau_w)^\gamma, w_{\min})$ combines the trajectory-validity mask $m_{i,t}$ with a soft motion-saliency term. Here $m_{i,t}$ excludes tracks affected by occlusion, invalid depth, or SpatialTrackerV2 confidence below 0.05; τ_w is the motion-magnitude threshold; κ controls the sigmoid sharpness; γ controls the saliency exponent; and $w_{\min}$ preserves a nonzero loss on static points. At inference, $\mathcal{F}$ is recovered from $F_0 \sim \mathcal{N}(0, I)$ by an explicit Euler integration of u_θ over a small number of flow steps and then mapped back to metric displacements using the inverse of the training normalization.

RoboTwin2.0 training protocol and full results. We train one multi-task policy on all 50 RoboTwin2.0 tasks, using 50 clean and 500 domain-randomized demonstrations per task (27,500 demonstrations in total). We follow the official RoboTwin2.0 data-collection and Easy/Hard evaluation protocols. We fine-tune all $\pi_{0.5}$ parameters from its public base checkpoint. GeomVLA uses a pretrained VLM backbone but no robot-action pretraining and is optimized only on these RoboTwin2.0 demonstrations. Both policies predict end-effector pose actions. Table 5 reports per-task success rates under the Easy and Hard randomization settings.

Two-seed stability check on CALVIN and LIBERO. We train one additional GeomVLA model on each benchmark. The CALVIN scores are 4.624 and 4.609 for seeds 0 and 42, and the LIBERO success rates are 98.4% and 98.0%. These differences are smaller than the principal ablation gaps in Table 2.

Evaluation of the 3D scene trajectory prediction horizon. We examine how the prediction horizon affects trajectory accuracy and downstream policy performance. Keeping all other components fixed, we train GeomVLA on the same CALVIN demonstrations using either the default horizon $H = 15$ or a longer horizon $H = 32$, and evaluate both trajectory predictors on held-out

Table 4: **Model configurations across simulation benchmarks and real-world experiments.** The 3D action denoiser shared attention layers are the action–scene fusion blocks that are shared by translation and rotation prediction. Rotation format is the policy rotation representation, with RoboTwin2.0 rotations converted to 6D internally; action horizon is the number of future actions predicted per policy query, and embodiment specifies the controlled arm setup.

	Module	CALVIN	LIBERO	RoboTwin2.0 – five-task ablations	Real-world
Trajectory Denoiser	Hidden dim	512	384	320	128
	Encoder layers	6	6	6	4
	Decoder layers	8	8	8	8
	Attention heads	16	8	8	8
	Batch size	128	128	128	128
	Optimizer	AdamW	AdamW	AdamW	AdamW
	Learning rate	1e-4	1e-4	1e-4	1e-4
	Iterations	20k	20k	20k	20k
Action Denoiser	Hidden dim	960	912	912	256
	Shared attention layers	16	16	16	16
	Attention heads	16	16	16	16
	Batch size	32	128	64	32
	Optimizer	AdamW	AdamW	AdamW	AdamW
	Learning rate	2e-5	5e-5	4e-5	2e-5
	Iterations	50k	80k	300k	50k
	Rotation format	Euler	Axis-angle	6D rotation	6D rotation
	Action horizon	10	10	16	8
	Embodiment	Single-arm	Single-arm	Bimanual	Single-arm

valid moving-point tracks over their shared first 15 predicted steps. SpatialTrackerV2 provides the pseudo-reference tracks. As shown in Table 6, increasing the horizon from $H = 15$ to $H = 32$ increases the mean error from 2.46 cm to 3.35 cm and the endpoint error from 4.61 cm to 6.43 cm. The CALVIN score also decreases from 4.624 to 4.272, suggesting that requiring the denoiser to predict farther into the future may make the trajectory-learning problem harder and weaken the motion representation used by the downstream policy.

Instruction conditioning for the 3D Scene Trajectory Denoiser. We examine whether the predicted trajectories capture instruction-conditioned motion rather than generic scene dynamics. We replace the correct task instruction with a mismatched instruction while keeping the visual observation unchanged. Because changing only the instruction changes the trajectory prediction and increases its error relative to the matched-task pseudo-reference, the denoiser is sensitive to language conditioning.

Control for model capacity. We replace the learned 3D Scene Trajectory Denoiser with a randomly initialized module of the same size, freeze it, and retrain the policy. The CALVIN score decreases from 4.624 to 4.443, indicating that the gain is associated with learned motion features rather than the added parameter count alone.

Inference latency. We measure the latency of one `model.step()` call during online CALVIN evaluation; each call predicts a chunk of 10 actions, all of which are executed before the next policy query. Measurements use CUDA-synchronized timers during PyBullet rollouts with EGL rendering on one NVIDIA L40S GPU. For each model, we evaluate four instruction sequences and discard the first 10 chunks as warm-up. GeomVLA without motion requires 219.0 ± 11.3 ms per chunk over 165 calls: 35.5 ms for the VLM encoder and 182.4 ms for the 3D action policy. Full GeomVLA requires 280.0 ± 14.0 ms over 155 calls: 37.2 ms for the VLM encoder, 37.5 ms for the trajectory field, and 202.1 ms for the action policy. Full GeomVLA therefore adds 61.0 ms per chunk, or 27.9% overhead. The trajectory field accounts for 61.5% of this increase, while most of the remainder arises from the slower action-policy stage. In both models, the action policy dominates total inference time.

Ablation variant descriptions. Table 8 provides detailed descriptions of the model variants used in the main-paper ablation study. These variants isolate the effects of 3D future-motion reasoning, 3D action denoising, motion-token extraction, and coordinate-frame alignment.

Table 5: **Per-task success rates on 50 RoboTwin2.0 tasks.** Each task is evaluated under both Easy and Hard domain randomization, with 50 rollouts per setting. We report success rates as percentages. **Average: $\pi_{0.5}$ Easy 75.9 / Hard 75.7, GeomVLA Easy 78.6 / Hard 76.2.**

Task	$\pi_{0.5}$		GeomVLA		Task	$\pi_{0.5}$		GeomVLA	
	Easy	Hard	Easy	Hard		Easy	Hard	Easy	Hard
Adjust Bottle	86	92	100	100	Place Can Basket	70	70	84	76
Beat Block Hammer	84	80	88	90	Place Cans Plasticbox	74	86	94	92
Blocks Ranking RGB	88	84	46	44	Place Container Plate	96	98	100	98
Blocks Ranking Size	52	38	24	32	Place Dual Shoes	62	64	84	86
Click Alarmclock	94	98	98	96	Place Empty Cup	90	90	100	96
Click Bell	98	98	100	100	Place Fan	88	84	88	86
Dump Bin Bigbin	84	90	72	68	Place Mouse Pad	44	36	88	90
Grab Roller	98	96	100	100	Place Object Basket	70	80	48	56
Handover Block	42	32	14	10	Place Object Scale	80	70	96	98
Handover Mic	42	52	72	38	Place Object Stand	86	94	96	98
Hanging Mug	16	14	6	4	Place Phone Stand	64	66	76	84
Lift Pot	88	88	32	18	Place Shoe	88	84	96	100
Move Can Pot	92	88	100	98	Press Stapler	70	78	70	78
Move Pillbottle Pad	80	88	96	94	Put Bottles Dustbin	34	38	34	18
Move Playingcard Away	74	76	98	96	Put Object Cabinet	56	60	62	48
Move Stapler Pad	56	64	72	76	Rotate QRcode	96	84	80	80
Open Laptop	90	92	94	92	Scan Object	62	56	62	60
Open Microwave	82	80	94	82	Shake Horizontally	96	98	100	100
Pick Diverse Bottles	86	84	86	84	Shake Bottle	96	98	100	100
Pick Dual Bottles	94	94	100	96	Stack Blocks Three	56	60	46	32
Place A2B Left	84	90	90	90	Stack Blocks Two	92	92	74	88
Place A2B Right	82	72	96	90	Stack Bowls Three	78	64	46	36
Place Bread Basket	74	68	86	86	Stack Bowls Two	88	98	84	82
Place Bread Skillet	72	66	92	80	Stamp Seal	88	74	96	96
Place Burger Fries	78	82	96	94	Turn Switch	56	58	72	72

Table 6: **Effect of trajectory-prediction horizon on CALVIN.** Errors are computed on held-out valid moving points against SpatialTrackerV2 pseudo-reference tracks over the first 15 future steps for both models. In this two-setting comparison, the longer training horizon yields higher trajectory error and lower downstream performance.

Horizon	Mean error (cm) ↓	Endpoint error (cm) ↓	Perf. ↑
$H = 15$	2.46	4.61	4.624
$H = 32$	3.35	6.43	4.272

Real-world task descriptions. We provide detailed descriptions of all eight real-world manipulation tasks, grouped by the primary capability they evaluate. Together, these tasks assess precise pose control, fine-grained small-object manipulation, color grounding, and multi-stage spatial reasoning. The number of demonstrations collected for each task is reported in parentheses.

Precision and orientation control.

- **Insert marker** (50 demos). The robot picks up a marker lying flat on the table and inserts it vertically into a cup. This task requires accurate geometric reasoning and precise end-effector orientation under tight spatial tolerances.
- **Stand bottle upright** (20 demos). The robot picks up a small bottle lying flat on the table, rotates it to an upright orientation, and places it stably on the table. The task requires a large end-effector rotation followed by controlled placement to prevent the bottle from falling.
- **Transfer marker** (20 demos). The robot picks up a marker standing inside one cup and inserts it into another. Because the marker pose varies across trials, the policy must adapt its gripper orientation for both grasping and insertion. It must also avoid lifting or knocking over the source cup while extracting the marker.

Table 7: **Instruction sensitivity of predicted 3D scene trajectories.** Endpoint error is measured in centimeters on held-out valid moving points against SpatialTrackerV2 pseudo-reference tracks at $H = 15$. Replacing the matched instruction with a mismatched instruction increases the error on all three benchmarks.

Benchmark	Matched ↓	Mismatched ↓	Increase
CALVIN	4.61	8.01	+3.40
LIBERO	9.12	14.56	+5.44
RoboTwin2.0	5.77	7.41	+1.64

Table 8: **Ablation model descriptions.** Detailed descriptions of the model variants evaluated in the ablation study. Each variant modifies one aspect of GeomVLA, including future-motion reasoning, action denoising, motion-token extraction, or coordinate-frame alignment, while the corresponding quantitative results are reported in the main paper.

Variant	Description
GeomVLA w/o motion	Removes the 3D Scene Trajectory Denoiser and directly predicts actions with the 3D action denoiser.
2DGeomVLA w/o motion	Uses a 2D image-based action denoiser without future-motion reasoning.
2DGeomVLA	Uses 2D future-motion reasoning with a 2D action denoiser.
2DGeomVLA LaMP-style	Uses image-plane (u, v) coordinates and metric depth for motion reasoning instead of metric 3D points.
GeomVLA w/ 2D motion	Uses image-space future motion with the 3D action denoiser.
GeomVLA w/ denoised trace	Conditions action denoising on the fully denoised 3D trajectory.
GeomVLA w/ denoised latent	Conditions action denoising on the fully denoised motion latent.
GeomVLA w/ partial denoising	Reads the motion latent at $\tau = 0.1$ instead of pure noise.
GeomVLA encoder-only	Uses only the trajectory-denoiser encoder, without velocity prediction.
GeomVLA w/ arbitrary frame	Expresses scene, motion, and action in a shared arbitrary rigid frame.
GeomVLA	Unifies scene, future motion, and action in a shared 3D frame.

Fine-grained small-object manipulation.

- **Stack blocks** (50 demos). The robot picks up a small block and stacks it on another block of identical size and shape. Successful execution requires accurate grasping, close alignment, and a controlled low-height release; otherwise, the upper block can easily slide off.
- **Uncap marker** (20 demos). The robot identifies and grasps the cap of a marker lying flat on the table and pulls it off. This task requires the policy to distinguish the cap from the marker body and precisely manipulate only the cap.

Color grounding and multi-stage spatial reasoning.

- **Place bottles** (50 demos). The robot places the green bottle into the green cup and then places the blue bottle into the blue cup. This two-stage task evaluates color grounding and sequential object placement.
- **Rank by color** (20 demos). The robot sequentially arranges three bottles from left to right, as viewed from the wrist camera, in pink–green–blue order. This three-stage, long-horizon task combines color grounding, ordered spatial reasoning, and sequential manipulation.
- **Place in triangle** (20 demos). Using the middle bottle as an anchor vertex, the robot sequentially moves the left and right bottles, as viewed from the wrist camera, so that the three bottles form a triangle. This two-stage task evaluates reasoning about relative spatial relationships and precise placement.

During evaluation, we vary distractors, goal objects, and object layouts across both seen and unseen settings. Figure 5 shows representative successful GeomVLA rollouts on the five real-world tasks trained with 20 demonstrations each. Each row presents a temporal sequence from one rollout, illustrating how the policy handles diverse challenges such as precise object reorientation, small-object grasping, partial occlusion, and multi-stage spatial reasoning. These qualitative examples complement the quantitative results by showing the execution process behind successful task completion.

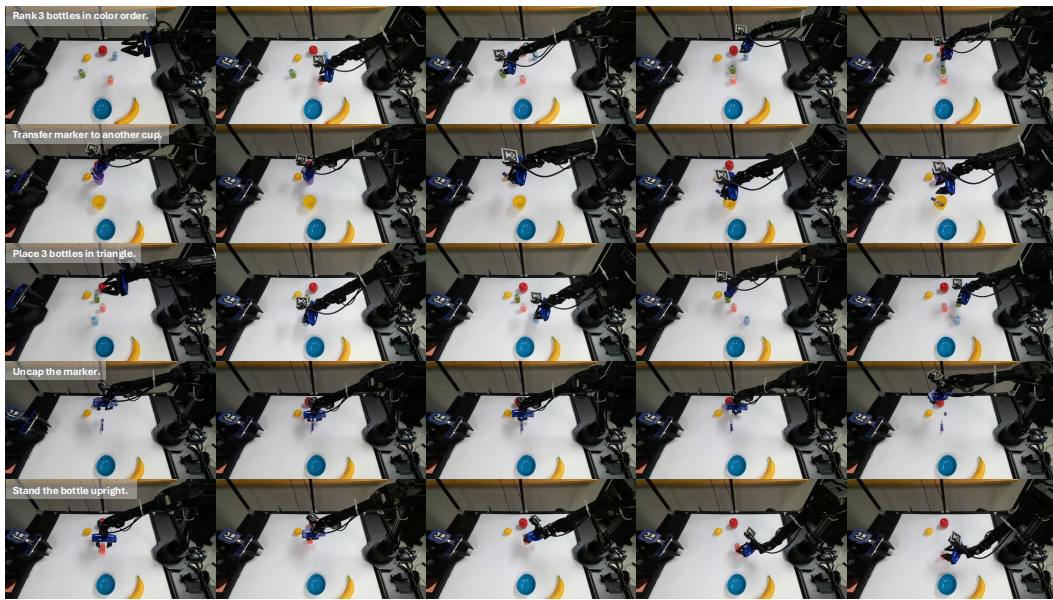


Figure 5: **Qualitative GeomVLA rollouts on five additional real-world tasks.** These tasks complement those shown in Figure 4. Each row shows a temporal sequence from one successful rollout. From top to bottom, the tasks are Rank by color, Transfer marker, Place in triangle, Uncap marker, and Stand bottle upright.